

GIT COMMANDS

1. Git Init

Definition:

Creates a new Git repository in the current folder.

Syntax:

```
git init
```

2. Git Config

Definition:

Used to configure Git settings such as username and email.

Syntax:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

3. Git Status

Definition:

Shows the current state of the repository, including modified, staged, and untracked files.

Syntax:

```
git status
```

4. Git Add

Definition:

Adds files to the staging area before committing.

Syntax:

```
git add filename  
git add .
```

5. Git Commit

Definition:

Saves the staged changes permanently in the local repository.

Syntax:

```
git commit -m "Commit message"
```

6. Git Push**Definition:**

Uploads local commits to the remote repository.

Syntax:

```
git push origin branch_name
```

7. Git Pull**Definition:**

Downloads the latest changes from the remote repository and merges them into the current branch.

Syntax:

```
git pull origin branch_name
```

8. Git Fetch --all**Definition:**

Downloads updates from all remote repositories without merging them.

Syntax:

```
git fetch --all
```

9. Git Branch**Definition:**

Creates, lists, or manages branches in the repository.

Syntax:

```
git branch  
git branch branch_name
```

Git Branch --all

Definition:

Displays all local and remote branches.

Syntax:

```
git branch --all
```

10. Git Checkout

Definition:

Switches to another branch or restores files from a previous commit.

Syntax:

```
git checkout branch_name
```

11. Git Switch

Definition:

Used only to switch between branches.

Syntax:

```
git switch branch_name
```

12. Git Merge

Definition:

Combines changes from one branch into another.

Syntax:

```
git merge branch_name
```

13. Git Rebase

Definition:

Moves or reapplies commits from one branch onto another branch to create a linear history.

Syntax:

```
git rebase branch_name
```

14. Git Restore

Definition:

Restores files to their previous state or removes changes.

Syntax:

```
git restore filename
```

15. Git Remote

Definition:

Manages remote repository connections.

Syntax:

```
git remote -v  
git remote add origin repository_URL
```

16. Git Log

Definition:

Displays the commit history of the repository.

Syntax:

```
git log
```

17. Git Diff

Definition:

Shows the differences between files, commits, or branches.

Syntax:

```
git diff
```

18. Git Reset (Soft, Mixed, Hard)

Soft Reset

Definition:

Moves the HEAD to a previous commit but keeps changes staged.

Syntax:

```
git reset --soft HEAD~1
```

Mixed Reset**Definition:**

Moves the HEAD and unstages changes, but keeps them in the working directory.

Syntax:

```
git reset --mixed HEAD~1
```

Hard Reset**Definition:**

Moves the HEAD and permanently removes all changes.

Syntax:

```
git reset --hard HEAD~1
```

19. Git Revert**Definition:**

Creates a new commit that reverses the changes of a previous commit.

Syntax:

```
git revert commit_id
```

20. Git Cherry-pick**Definition:**

Applies a specific commit from one branch to another.

Syntax:

```
git cherry-pick commit_id
```

21. Fork**Definition:**

Creates a copy of someone else's remote repository into your own GitHub account.

22. Clone

Definition:

Downloads a remote repository to your local computer.

Syntax:

```
git clone repository_URL
```

Important Comparison

Fork vs Clone

Fork

Creates a copy of another person's repository into your GitHub account.

Done on GitHub.

Used when you don't have direct access to the original repository.

No Git command (GitHub feature).

Clone

Creates a local copy of a remote repository on your computer.

Done using Git.

Used to download a repository for development.

Syntax:

```
git clone repository_URL
```

Pull vs Push

Git Pull

Downloads changes from the remote repository.

Updates the local repository.

Direction: Remote → Local.

Equivalent to Fetch + Merge.

Git Push

Uploads local commits to the remote repository.

Updates the remote repository.

Direction: Local → Remote.

Shares your changes with others.

Fetch vs Pull

Git Fetch

Downloads changes from the remote repository.

Does not merge changes.

Safe for reviewing updates.

Working files remain unchanged.

Git Pull

Downloads and merges changes.

Automatically updates the current branch.

Changes the working directory.

Equivalent to Fetch + Merge.

Checkout vs Switch

Git Checkout

Switches branches.

Can restore files and commits.

Older Git command.

Has multiple purposes.

Git Switch

Switches branches only.

Cannot restore files.

Newer Git command.

Simpler and easier to use.

Status vs Log

Git Status

Shows the current repository status.

Displays modified, staged, and untracked files.

Shows present changes.

Used before committing.

Git Log

Shows the commit history.

Displays commit ID, author, date, and message.

Shows previous commits.

Used to track project history.

Diff vs Status

Git Diff

Shows line-by-line changes.
Compares file contents.
Displays exact modifications.
Used to review changes.

Git Status

Shows file status only.
Does not show content changes.
Displays modified and untracked files.
Shows repository state.

Merge vs Rebase

Git Merge

Combines two branches.
Creates a merge commit.
Preserves branch history.
Easy and safe for collaboration.

Git Rebase

Moves commits onto another branch.
Usually does not create a merge commit.
Creates a clean, linear history.
Used to keep commit history clean.

Init vs Clone

Git Init

Creates a new empty Git repository.
Used for a new project.
Does not download files.
Starts version control.

Git Clone

Copies an existing repository.
Used for an existing project.
Downloads all files and commit history.
Creates a local working copy.

Revert vs Reset

Git Revert

Creates a new commit to undo changes.

Does not remove commit history.

Safe for shared repositories.

Used after changes are pushed.

Git Reset

Removes or moves commits.

Can remove commit history.

Best for local changes before sharing.

Used to undo local commits.